



پردیس دانشکده‌های فنی

دانشگاه تهران



رباتیک پیشرفته - نیمسال دوم ۱۴۰۳-۴

فاز چهارم پروژه

Path Planning and Navigation

نگارش:

علی خیری 810603071

معین رضایی 810602081

محمد رضا توکلی 159403003

استاد درس:

دکتر نصیری - دکتر مرادی

تابستان ۱۴۰۴

۱ مقدمه

۱-۱ برنامه ریزی مسیر (Path Planning)

برنامه‌ریزی مسیر (Path Planning) به معنای یافتن یک مسیر ایمن و بهینه از موقعیت فعلی ربات تا مقصد موردنظر است، به گونه‌ای که ربات بدون برخورد با موانع بتواند حرکت کند. این فرآیند یکی از بخش‌های اصلی ناوبری ربات‌های متحرک است و بین مرحله درک محیط (با استفاده از سنسورها و نقشه‌سازی) و مرحله کنترل حرکت قرار می‌گیرد. در واقع، Path Planning تعیین می‌کند که ربات از چه مسیری حرکت کند تا علاوه بر رسیدن به هدف، محدودیت‌های محیطی و دینامیکی خود را نیز در نظر داشته باشد.

برای حل این مسئله، روش‌های مختلفی به کار می‌رود. الگوریتم‌های کلاسیک مانند دایکسترا و A^* مسیر بهینه را روی نقشه محاسبه می‌کنند، در حالی که روش‌های نمونه‌گیری مانند RRT در محیط‌های پیچیده مؤثرترند. در سال‌های اخیر نیز استفاده از یادگیری ماشین و هوش مصنوعی به ربات‌ها امکان داده تا در محیط‌های ناشناخته و پویا تصمیم‌گیری بهتری داشته باشند. به طور خلاصه، برنامه‌ریزی مسیر به ربات می‌گوید نه تنها کجا برود بلکه چگونه به آنجا برسد.

۲-۱ انواع برنامه ریزی مسیر

❖ روش‌های گرافی (Graph-based Planning)

در روش‌های گرافی، محیط به شکل گراف یا شبکه مدل می‌شود و مسیر بین نقاط با الگوریتم‌هایی مثل Dijkstra یا A^* پیدا می‌شود. این الگوریتم‌ها مسیر کوتاه‌ترین یا کم‌هزینه‌ترین را تضمین می‌کنند و برای محیط‌های از قبل شناخته‌شده مناسب هستند. اما مشکل این روش این است که محاسبات زیادی نیاز دارد و در محیط‌های بزرگ یا پویا زمان‌بر است. همچنین نیاز به نقشه دقیق دارد، بنابراین بیشتر در محیط‌های ثابت یا نیمه‌ثابت استفاده می‌شود.

❖ روش‌های نمونه‌برداری (Sampling-based Planning)

روش‌هایی مانند RRT (Rapidly-exploring Random Tree) و PRM (Probabilistic Roadmap) در این دسته قرار دارند. این الگوریتم‌ها به جای جستجوی کامل محیط، نقاطی از محیط را به صورت تصادفی نمونه‌برداری می‌کنند و مسیر بین آن‌ها را پیدا می‌کنند. مزیت این روش‌ها این است که در محیط‌های پیچیده و شلوغ سریع

عمل می‌کنند و نیاز به مدل کامل محیط ندارند. اما تضمینی برای پیدا کردن مسیر کاملاً بهینه وجود ندارد و مسیر نهایی ممکن است طولانی یا پرپیچ‌وخم باشد.

❖ روش‌های مبتنی بر پتانسیل (Potential Field)

در این روش، هدف به عنوان یک نیروی جاذبه و موانع به عنوان نیروهای دافعه در نظر گرفته می‌شوند. ربات با دنبال کردن ترکیب این نیروها حرکت می‌کند و به سمت هدف می‌رود. این روش ساده و سریع است و محاسبات سبک دارد. مشکل اصلی روش پتانسیل، گیر افتادن ربات در مینیمم‌های محلی است؛ یعنی ممکن است ربات در نقطه‌ای توقف کند که نیروهای جاذبه و دافعه همدیگر را خنثی کرده‌اند و دیگر قادر به ادامه حرکت نیست. برای رفع این مشکل معمولاً الگوریتم‌های اصلاحی یا ترکیبی استفاده می‌شوند.

❖ روش‌های هوش مصنوعی (AI-based Planning)

روش‌های نوین برنامه‌ریزی مسیر از یادگیری ماشین و یادگیری تقویتی استفاده می‌کنند. ربات با تجربه و آموزش می‌تواند بهترین مسیر را در محیط‌های ناشناخته و پویا پیدا کند. شبکه‌های عصبی می‌توانند الگوهای حرکت را یاد بگیرند و ربات را قادر سازند در شرایط غیرقابل پیش‌بینی تصمیمات بهینه اتخاذ کند. این روش انعطاف بالایی دارد و برای محیط‌های پیچیده بسیار مناسب است، اما نیاز به داده زیاد و توان پردازشی بالا دارد. به همین دلیل بیشتر در ربات‌های هوشمند پیشرفته و پروژه‌های تحقیقاتی استفاده می‌شود.

۳-۱ ناوبری (Navigation)

ناوبری (Navigation) در ربات‌های متحرک به این معناست که ربات بتواند راه خود را در محیط پیدا کند و بدون برخورد با موانع به سمت هدف حرکت کند. برای این کار، ربات باید بداند کجاست، محیط اطرافش چه شکلی است و چطور باید حرکت کند تا به مقصد برسد. اگر ربات سیستم ناوبری نداشته باشد، حرکتش بی‌هدف خواهد بود و نمی‌تواند وظایفش را درست انجام دهد.

یکی از بخش‌های مهم ناوبری، پیدا کردن موقعیت ربات یا همان مکان‌یابی است. ربات با استفاده از سنسورها مثل دوربین، لیزر یا انکودر چرخ‌ها می‌فهمد در کدام قسمت از محیط قرار دارد. علاوه بر این، باید بتواند موانع و اجسام اطرافش را شناسایی کند تا هنگام حرکت به آن‌ها برخورد نکند. این اطلاعات کمک می‌کند ربات دیدی کلی از محیط داشته باشد.

۴-۱ انواع ناوبری

❖ ناوبری محلی

ناوبری محلی به ربات اجازه می‌دهد تنها به محیط اطراف خود توجه کند و تصمیم‌های لحظه‌ای برای حرکت بگیرد. ربات با استفاده از حسگرهایی مانند لیدار، دوربین و انکودر چرخ‌ها موانع نزدیک را شناسایی کرده و مسیر خود را به گونه‌ای تغییر می‌دهد که از برخورد جلوگیری شود. این روش سرعت بالایی دارد و برای محیط‌های پویا که شرایط آن‌ها مرتباً تغییر می‌کند، مناسب است. با وجود سرعت و سادگی، ناوبری محلی محدودیت‌هایی دارد. ربات فقط دید کوتاه‌مدت دارد و ممکن است در بن‌بست گیر کند یا مسیر طولانی‌تری نسبت به حالت بهینه انتخاب کند. به همین دلیل، معمولاً ناوبری محلی به تنهایی استفاده نمی‌شود و به صورت مکمل ناوبری سراسری به کار گرفته می‌شود.

❖ ناوبری سراسری

ناوبری سراسری بر پایه نقشه کلی محیط انجام می‌شود. در این حالت ربات از قبل یک نقشه کامل یا تقریبی از محیط دارد و مسیر از مبدا تا مقصد مشخص است. الگوریتم‌های برنامه‌ریزی مسیر در این نوع ناوبری نقش کلیدی دارند و باعث می‌شوند ربات راه کوتاه‌تر و کم‌هزینه‌تری انتخاب کند. این روش دقت بالایی دارد اما در محیط‌های پویا که موانع ناگهانی ظاهر می‌شوند، انعطاف کمی دارد. در چنین شرایطی، ربات نیاز به بروزرسانی مداوم نقشه دارد یا باید از ناوبری ترکیبی استفاده کند تا در لحظه تصمیمات مناسب بگیرد.

❖ ناوبری ترکیبی

ناوبری ترکیبی به ترکیب دو روش محلی و سراسری گفته می‌شود. در این حالت، ربات هم مسیر کلی از قبل مشخص دارد و هم در حین حرکت، با استفاده از داده‌های حسگرها مسیر را اصلاح می‌کند. این رویکرد، هم دقت مسیر و هم انعطاف لازم برای مقابله با موانع جدید را فراهم می‌کند. ناوبری ترکیبی این است که می‌توان در محیط‌های واقعی، پیچیده و پویا از آن استفاده کرد. خودروهای خودران و ربات‌های خدماتی پیشرفته معمولاً از این روش بهره می‌برند، زیرا ترکیب دید بلندمدت نقشه و واکنش کوتاه‌مدت حسگرها باعث عملکرد بهتر ربات می‌شود.

۲ مروری بر الگوریتم A^*

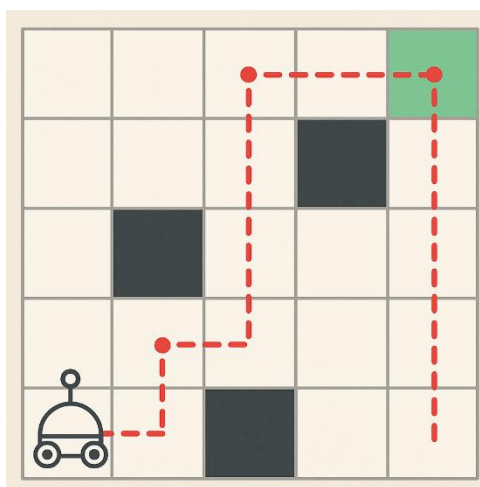
الگوریتم A^* یکی از معروف‌ترین الگوریتم‌های برنامه‌ریزی مسیر در رباتیک و هوش مصنوعی است. هدف آن پیدا کردن کوتاه‌ترین مسیر بین یک نقطه شروع و یک نقطه هدف در محیط‌های دارای موانع است A^* . با ترکیب دو مفهوم هزینه واقعی حرکت تاکنون و تخمین هزینه باقی‌مانده تا هدف، مسیر بهینه را پیدا می‌کند. این الگوریتم به گونه‌ای طراحی شده که هم دقیق باشد و هم نسبت به جستجوی کامل محیط سریع‌تر عمل کند.

برای هر گره (یا نقطه) در محیط، الگوریتم یک مقدار $f(n)$ محاسبه می‌کند که مجموع هزینه واقعی تا آن گره $g(n)$ و تخمین فاصله تا مقصد $h(n)$ است:

$$f(n) = g(n) + h(n)$$

الگوریتم ابتدا گره شروع را در یک لیست باز (Open List) قرار می‌دهد و گره‌هایی که بررسی شده‌اند را در لیست بسته (Closed List) ذخیره می‌کند. در هر مرحله، گره با کمترین مقدار $f(n)$ از لیست باز انتخاب شده و همسایگان آن بررسی می‌شوند تا مسیر بهینه ساخته شود.

مزیت اصلی A^* این است که اگر مسیر وجود داشته باشد، حتماً آن را پیدا می‌کند و کوتاه‌ترین مسیر را تضمین می‌کند. این الگوریتم در مسیریابی ربات‌ها، بازی‌های رایانه‌ای و سیستم‌های خودران کاربرد فراوان دارد. محدودیت آن مصرف حافظه نسبتاً بالاست، زیرا تمام گره‌های بررسی‌شده باید ذخیره شوند، اما نسخه‌های بهینه‌شده مثل Memory-bounded A^* می‌توانند این مشکل را کاهش دهند و الگوریتم را برای محیط‌های بزرگ و پویا مناسب کنند.



شکل ۱ الگوریتم A^*

۳ ساختار کلی الگوریتم پیاده شده

در این پروژه، هدف طراحی مسیر بهینه برای ربات‌های متحرک با استفاده از الگوریتم A^* مورد بررسی قرار گرفته است. برای رسیدن به این هدف، مراحل مختلفی پیاده‌سازی می‌شوند تا ربات بتواند به صورت خودکار مسیر خود را در محیط شناسایی شده برنامه‌ریزی و دنبال کند.

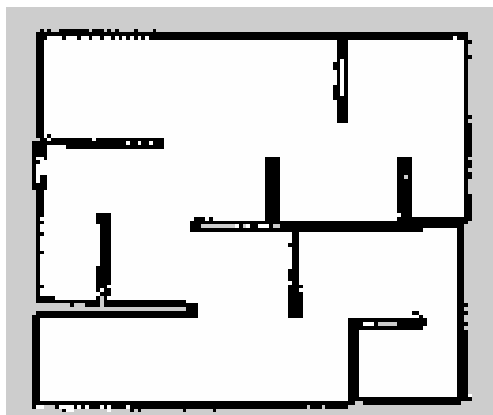
ابتدا با استفاده از الگوریتم Gmapping و ربات TurtleBot، نقشه‌ای از محیط ایجاد می‌شود. این نقشه، پایه‌ای برای برنامه‌ریزی مسیر و تصمیم‌گیری ربات فراهم می‌کند. سپس با پردازش این نقشه، ماتریس باینری از روی تصویر ایجاد می‌شود که مناطق قابل عبور و موانع به صورت واضح مشخص می‌شوند.

پس از آماده‌سازی ماتریس باینری، الگوریتم A^* برای طراحی مسیر بهینه بین نقطه شروع و مقصد به کار گرفته می‌شود. در نهایت، با استفاده از مسیر تولید شده، ربات قادر به ناوبری خودکار در محیط خواهد بود و می‌تواند به صورت هوشمندانه از موانع عبور کند و مسیر کوتاه‌ترین یا کم‌هزینه‌ترین را دنبال نماید.

۳-۱ تولید نقشه محیط

SLAM یک تکنیک کلیدی در رباتیک است که به ربات‌ها اجازه می‌دهد بدون داشتن نقشه از قبل، محیط اطراف خود را شناسایی کنند و همزمان موقعیت خود را در همان محیط تخمین بزنند. در این پروژه، از ربات TurtleBot3 و حسگر فاصله‌یابی LIDAR استفاده شده است تا الگوریتم Gmapping بتواند داده‌های محیط را جمع‌آوری و پردازش کند. به صورت شهودی، وقتی ربات در محیط حرکت می‌کند، LIDAR فاصله تا موانع و ویژگی‌های محیط را اندازه‌گیری می‌کند و این اطلاعات به Gmapping داده می‌شود. با ترکیب داده‌های سنسور و تخمین موقعیت ربات، یک نقشه دو بعدی از محیط تولید می‌کند و به تدریج تصویر کامل محیط شکل می‌گیرد. این روش به ربات امکان می‌دهد محیط‌های ناشناخته را کاوش کرده و از برخورد با موانع جلوگیری کند.

در طول فرآیند، نقشه در RViz به صورت بصری نمایش داده می‌شود تا کاربر بتواند روند نقشه‌برداری را مشاهده کند و از صحت داده‌ها اطمینان حاصل کند. پس از پایان کاوش، نقشه نهایی ذخیره می‌شود تا در مراحل بعدی، مانند طراحی مسیر با الگوریتم A^* و ناوبری خودکار ربات مورد استفاده قرار گیرد. با استفاده از این نقشه، ربات می‌تواند موقعیت خودش را دقیق تشخیص دهد و مسیرهای بهینه برای رسیدن به مقصد را برنامه‌ریزی کند.



شکل ۲ نقشه تولید شده از SLAM

۲-۳ تولید ماتریس از نقشه

این الگوریتم ابتدا یک تصویر از محیط را باز کرده و آن را به حالت خاکستری تبدیل می‌کند تا شدت روشنایی هر پیکسل مشخص شود. سپس با استفاده از یک آستانه‌گذاری ساده (Thresholding)، تصویر را به حالت باینری تبدیل می‌کند؛ یعنی پیکسل‌های روشن‌تر به مسیر آزاد (۰) و پیکسل‌های تیره‌تر به مانع (۱) تبدیل می‌شوند. به این ترتیب، تصویر رنگی یا خاکستری محیط به یک ماتریس دو بعدی شامل فقط اعداد ۰ و ۱ تبدیل می‌شود که هر خانه آن نشان‌دهنده وضعیت یک سلول در محیط است.

پس از باینری کردن، تصویر به اندازه دلخواه تغییر اندازه می‌شود تا پردازش‌های بعدی مانند مسیر یابی راحت‌تر انجام شود و سپس به آرایه عددی تبدیل می‌شود. این آرایه می‌تواند ذخیره شده و یا برای نمایش بصری با استفاده از نمودار نشان داده شود. در نهایت، این ماتریس باینری به ربات امکان می‌دهد مسیرهای آزاد و موانع را شناسایی کرده و از آن برای الگوریتم‌های Path Planning و ناوبری خودکار استفاده کند.

۳-۳ الگوریتم برنامه ریزی مسیر

الگوریتم A^* یکی از رایج‌ترین و موثرترین روش‌ها برای پیدا کردن مسیر بهینه در محیط‌های گسسته مانند نقشه‌های باینری است. ایده اصلی این الگوریتم ترکیب دو مفهوم مهم است: هزینه واقعی طی شده تا یک خانه (G-cost) و تخمین فاصله تا مقصد یا با جمع این دو مقدار، الگوریتم خانه‌هایی را که احتمال دارد سریع‌تر به هدف برسند، در اولویت قرار می‌دهد و به این ترتیب مسیر کوتاه‌ترین یا کم‌هزینه‌ترین مسیر ممکن از نقطه شروع تا مقصد پیدا می‌شود.

در این برنامه، ابتدا نقشه محیط با استفاده از تابع $\text{map}()$ بارگذاری می‌شود که یک ماتریس دو بعدی شامل ۰ (مسیر آزاد) و ۱ (مانع) ایجاد می‌کند. کاربر مختصات شروع و مقصد و شعاع ربات را وارد می‌کند تا اطمینان

حاصل شود مسیر انتخاب شده با موانع برخورد نکند. سپس الگوریتم A^* یک لیست اولویت دار ایجاد می کند که در آن خانه های بالقوه مسیر همراه با هزینه پیش بینی شده ذخیره می شوند. در هر مرحله، خانه ای که کمترین مجموع هزینه واقعی و تخمینی دارد از لیست خارج می شود و به عنوان خانه فعلی بررسی می شود. برای هر خانه فعلی، الگوریتم خانه های مجاور در هشت جهت ممکن را چک می کند. اگر خانه ای قابل عبور باشد و قبلاً بازدید نشده باشد، هزینه رسیدن به آن محاسبه شده و اولویت آن برای اضافه شدن به Heap تعیین می شود. همچنین با بررسی شعاع ربات (walkable) تضمین می شود که مسیر انتخاب شده به اندازه کافی از موانع فاصله داشته باشد. این فرآیند تا زمانی ادامه پیدا می کند که خانه مقصد پیدا شود یا هیچ مسیر قابل قبولی باقی نمانده باشد.

پس از رسیدن به مقصد، مسیر واقعی با استفاده از اطلاعات والد هر خانه (Parent Dictionary) بازسازی می شود. این بازسازی از مقصد به شروع انجام شده و سپس مسیر برعکس می شود تا ترتیب درست حرکت ربات را نشان دهد. در نهایت، مسیر به صورت یک لیست از مختصات ارائه می شود و می تواند روی نقشه نمایش داده یا برای کنترل ربات و ناوبری خودکار استفاده شود. به صورت کلی، الگوریتم A^* مسیرهای بالقوه را با هوشمندی جستجو می کند و همواره تلاش می کند به کمترین هزینه و کوتاه ترین مسیر برسد، در حالی که از موانع عبور نمی کند و ایمنی ربات حفظ می شود.

۴-۳ الگوریتم ناوبری ربات

الگوریتم ناوبری ربات، هدفش حرکت دادن ربات از نقطه شروع به مقصد مشخص شده است، به طوری که مسیر دنبال شده ایمن و قابل کنترل باشد. ابتدا ربات مختصات و جهت فعلی خود را با استفاده از اطلاعات ادورمتر (Odometry) دریافت می کند. این داده ها شامل موقعیت x ، y و زاویه چرخش ربات (Yaw) است. این اطلاعات به الگوریتم اجازه می دهد که همیشه بداند ربات در کجای محیط قرار دارد و به چه سمتی نگاه می کند.

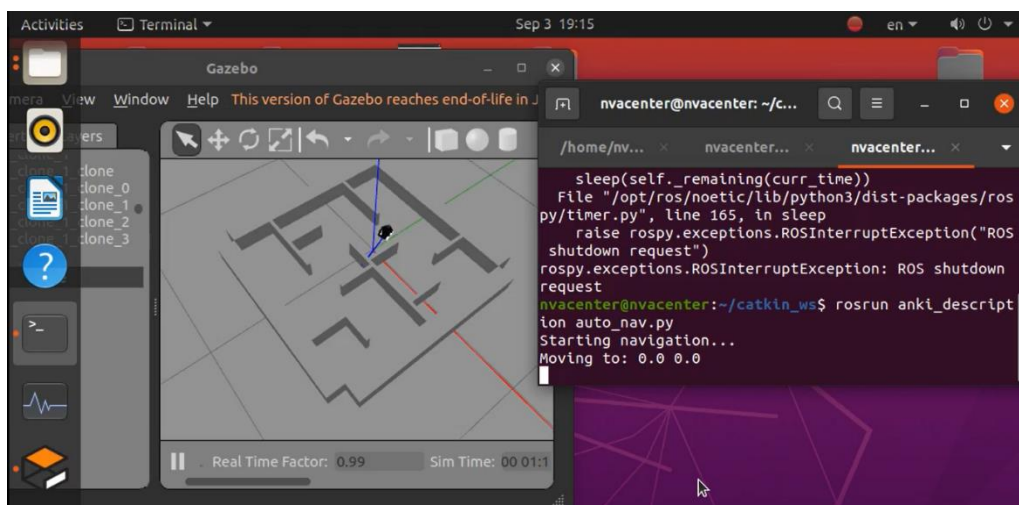
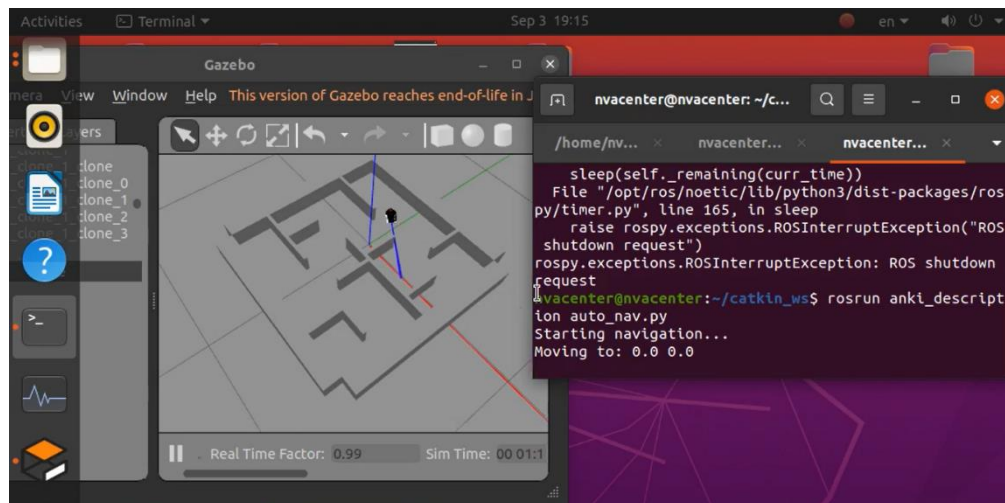
سپس برای هر نقطه از مسیر برنامه ریزی شده الگوریتم دو بخش کنترل اصلی دارد: کنترل خطی و کنترل زاویه ای. کنترل خطی میزان سرعت حرکت به سمت مقصد را تعیین می کند و کنترل زاویه ای ربات را به سمت هدف می چرخاند. اگر زاویه ربات با زاویه لازم برای رسیدن به مقصد اختلاف زیادی داشته باشد، ربات ابتدا می چرخد و حرکت مستقیم ندارد تا از انحراف مسیر جلوگیری شود. همچنین سرعت حرکت محدود می شود تا حرکت نرم و ایمن باشد. این فرآیند به صورت چرخه ای و مداوم انجام می شود تا ربات هر لحظه مسیرش را تصحیح کند و به تدریج به نقطه هدف برسد.

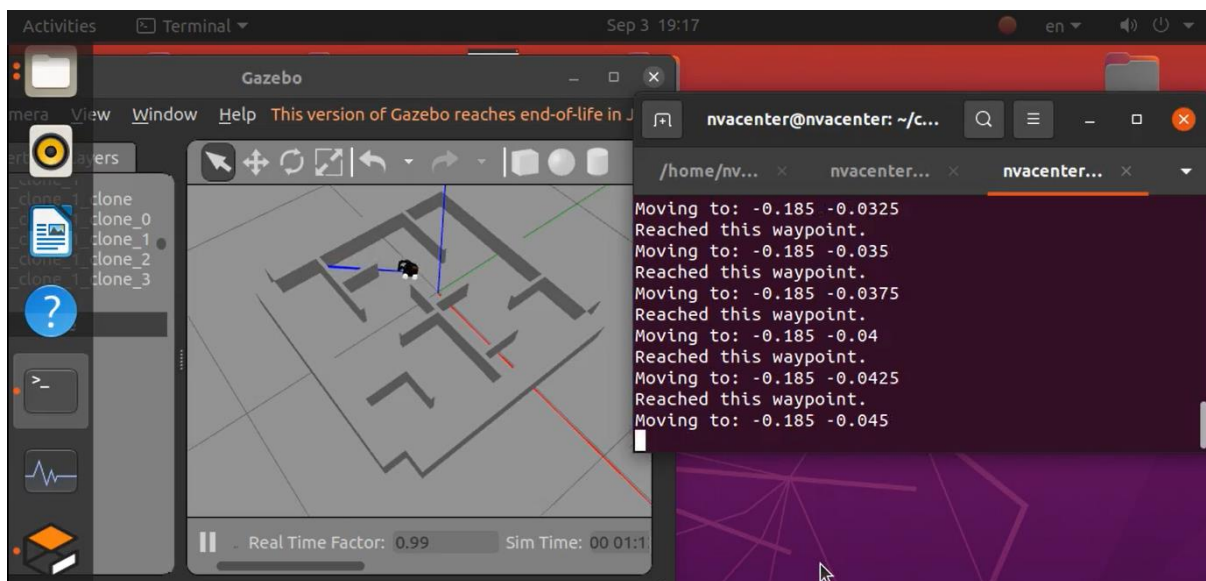
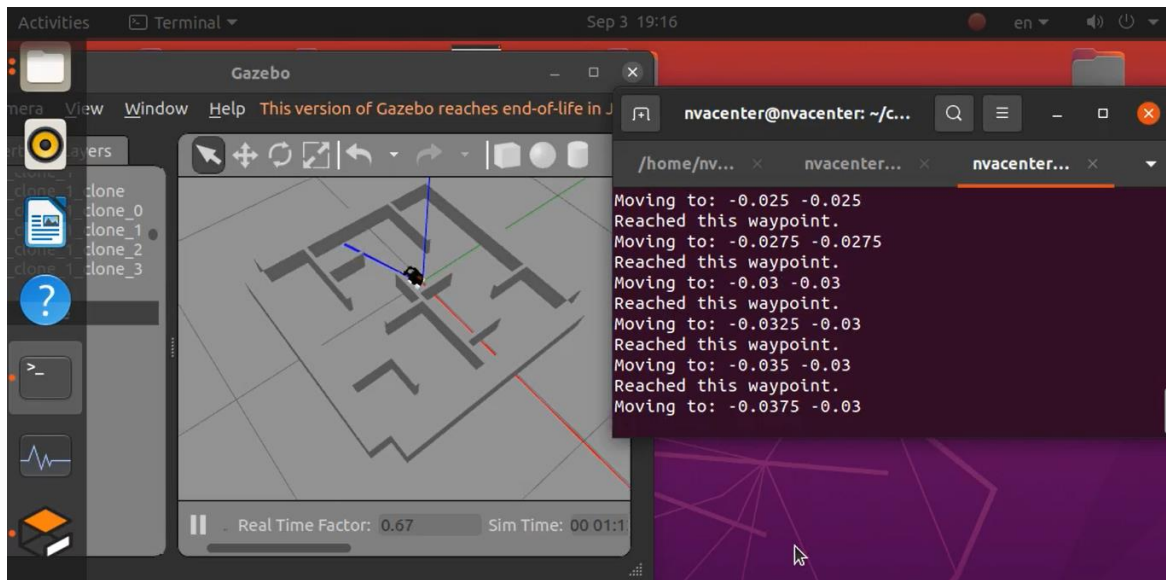
در نهایت، الگوریتم مسیر واقعی ربات را ذخیره و می تواند به صورت گرافیکی نشان دهد. این مسیر شامل موقعیت های پشت سر هم ربات از شروع تا پایان است و کمک می کند تا کاربر بتواند بررسی کند ربات چگونه

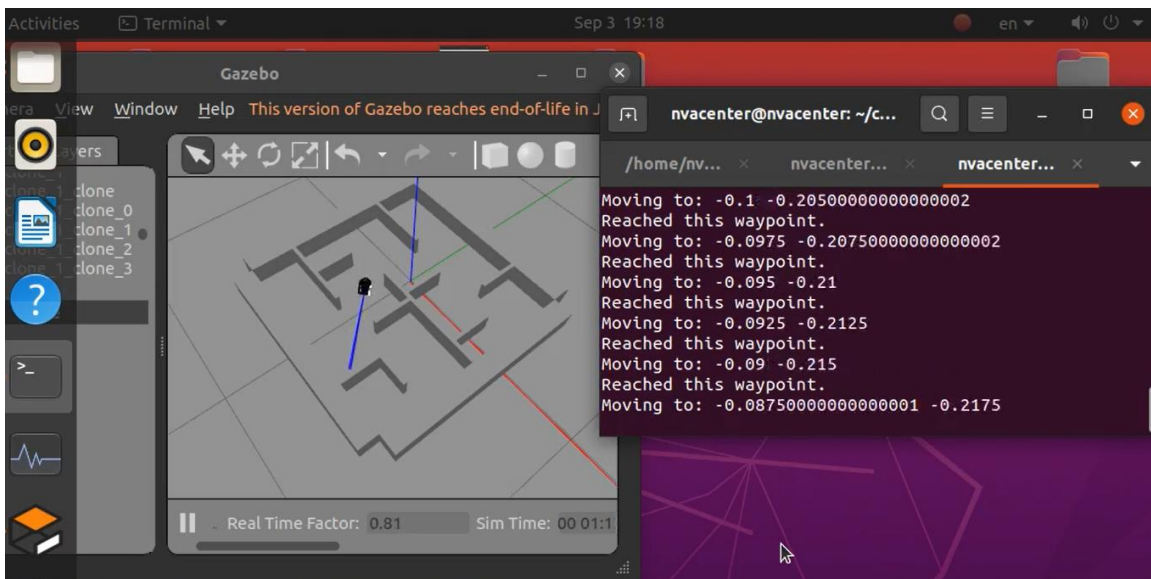
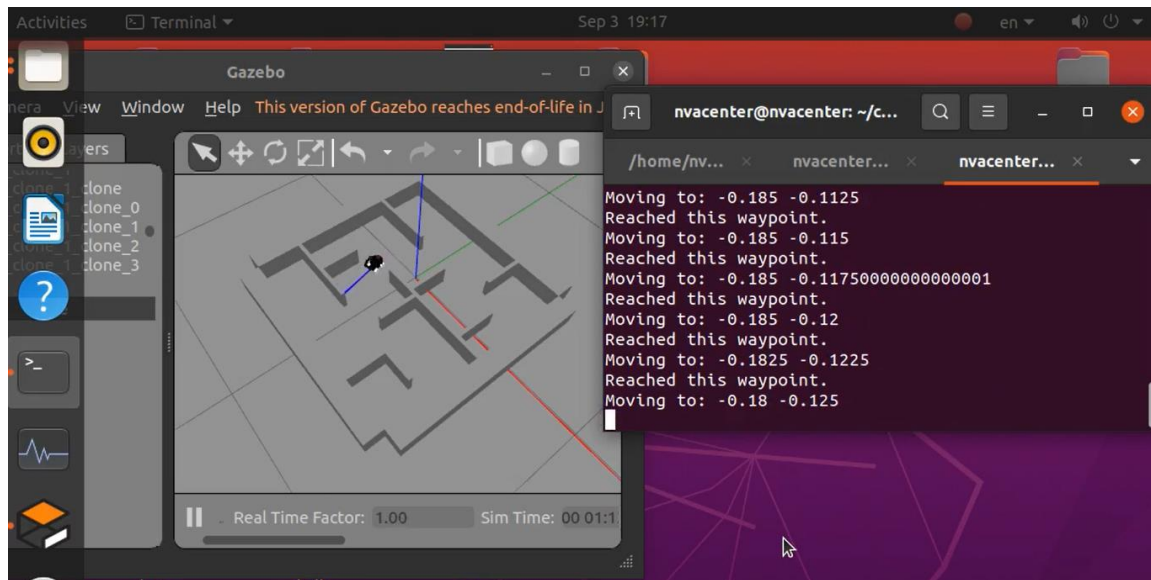
مسیر برنامه‌ریزی شده را دنبال کرده است. به طور خلاصه، این الگوریتم ترکیبی از خواندن موقعیت فعلی، تنظیم سرعت و زاویه، و پیروی از مسیر پیش‌بینی شده است تا ربات به صورت خودکار و ایمن به مقصد برسد.

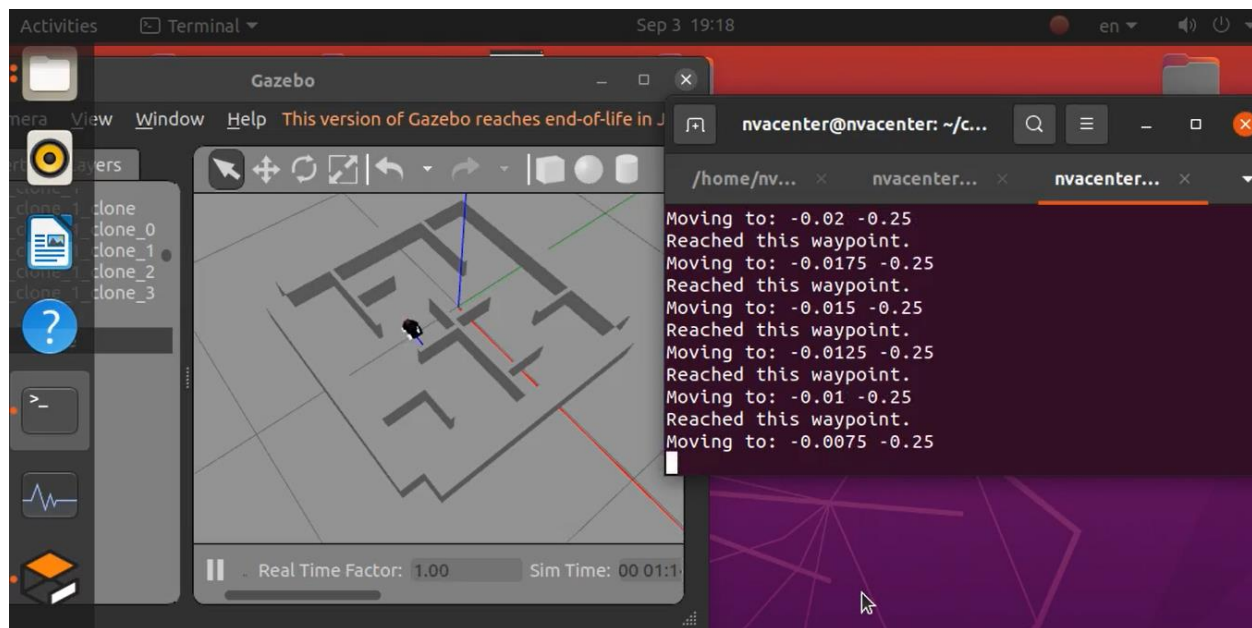
۴ نتایج و ارزیابی الگوریتم

ربات ابتدا مجموعه‌ای از نقاط را به عنوان مسیر طراحی می‌کند و همانطور که گفته شد ربات در ۸ جهت مختلف بر روی این گریدها قابل حرکت است. ما دو موقعیت (100,430) و (100,330) که یک مسیر صاف به موازات محور y است را به ربات دادیم و در میان این مسیر نیز یک دیوار قرار دارد. مشاهده کردیم که مطابق تصویر زیر ربات ابتدا خود را به نقطه $\bullet$ و $\bullet$ که شروع مسیر هست می‌آورد و سپس از این نقطه با ایجاد یک مسیر شکسته مانع را دور می‌زند.

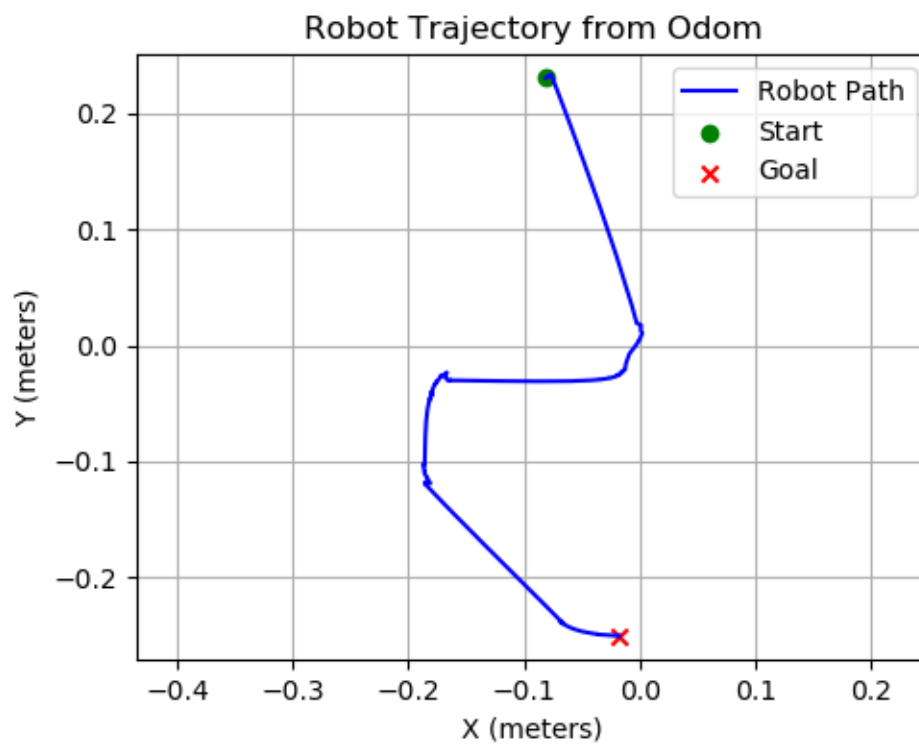








و در نهایت ربات با طی کردن مسیر زیر به مقصد خود رسید:



۵ پیشنهادات آینده

استفاده از الگوریتم D^* برای مسیر یابی دینامیک:

الگوریتم D^* یک نسخه پیشرفته از الگوریتم‌های مسیر یابی مانند A^* است که مخصوص محیط‌های پویا طراحی شده است. در حالی که A^* مسیر بهینه را بر اساس نقشه ثابت پیدا می‌کند، D^* قادر است به‌روزرسانی‌های محیطی را در طول حرکت ربات پردازش کند. این به این معنی است که اگر ربات در مسیر خود با مانع جدید یا تغییر محیط مواجه شود، الگوریتم به سرعت مسیر را بازنگری کرده و مسیر جدید بهینه را محاسبه می‌کند. استفاده از D^* می‌تواند ناوبری ربات را در محیط‌های واقعی و غیرقابل پیش‌بینی بسیار انعطاف‌پذیر کند و امکان حرکت امن و بدون توقف در محیط‌های پیچیده را فراهم نماید.

استفاده از سیستم‌های بینایی برای تشخیص موانع:

به جای تکیه صرف بر نقشه باینری یا حسگرهای محدود مانند LIDAR، می‌توان از دوربین‌ها و الگوریتم‌های بینایی ماشین (Vision-Based Obstacle Detection) برای تشخیص موانع استفاده کرد. با پردازش تصاویر و ویدئوهای محیط، ربات قادر خواهد بود موانع متحرک و ایستا را شناسایی کرده و مسیر خود را بر اساس آن‌ها بازسازی کند. این رویکرد به ویژه در محیط‌هایی که موانع تغییر می‌کنند یا اشیاء کوچک و غیرقابل شناسایی با حسگرهای سنتی وجود دارند، بسیار موثر است. ترکیب بینایی با الگوریتم‌های مسیر یابی پویا مانند D^* می‌تواند ربات را قادر سازد به صورت کاملاً هوشمند و خودکار در محیط‌های واقعی حرکت کند.